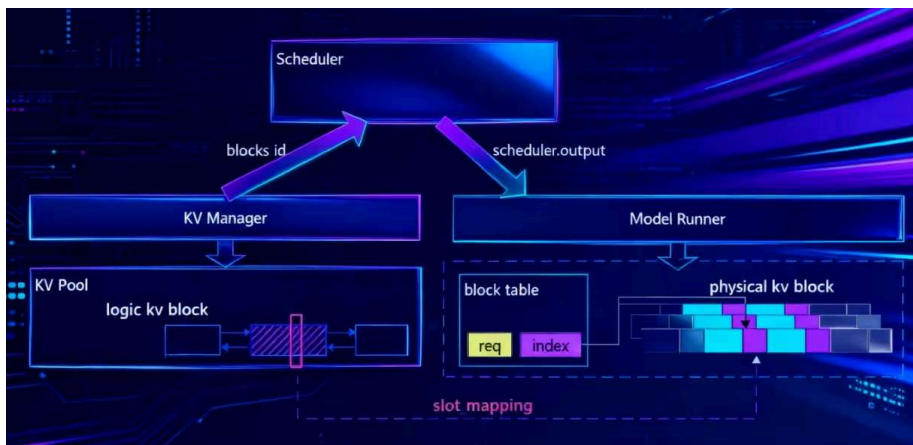




高效推理的核心：vLLM V1 KV cache 管理机制剖析



kaiyuan
大模型话题下的优秀答主

已关注

收录于 · LLM 推理基础与框架 >

128 人赞同了该文章 >

引言：KV cache的管理是vLLM框架最关键内容之一，在框架升级到V1后其逻辑进行了一次大的调整。为更好的了解KV cache的管理逻辑，本文结合代码(v0.10.2版本)，从整体架构到关键细节进行讲解，涵盖逻辑层、物理层以及两者间的联系，帮助使用者/开发者对整体结构有个基本了解。

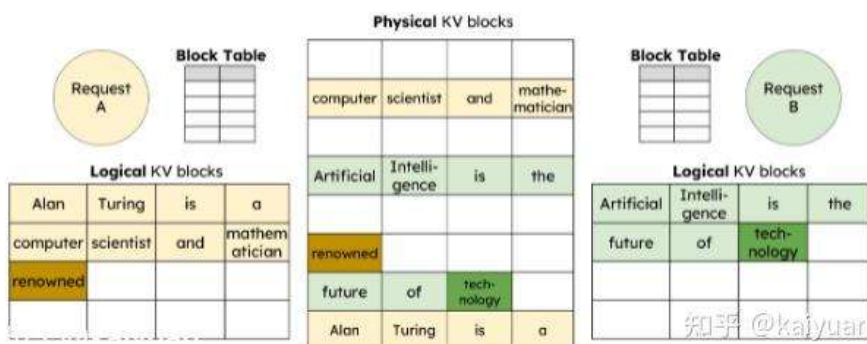
更多内容: [LLM推理知识指南---kaiyuan](#)

1 整体框架

1.1 基本原理

KV cache管理逻辑以PagedAttention为基础进行构建，分了逻辑层与物理层，该方式类似于操作系统的虚拟内存（[virtual memory](#)⁺）管理。虽然vLLM版本在快速迭代更新，但这个基础的逻辑保持一致，所以学习PagedAttention是了解KV cache管理的第一步。

PagedAttention的核心逻辑是将Attention运算中的KV值按照虚拟映射的方式管理起来，如下图所示。图中有两个请求request A和B，他们各自拥有自己的逻辑块（logical kv blocks），通过对应的映射表（block table）找到每个词在物理块（physical kv blocks）中的位置。



关于作者

Page

赞同 128

12 条评论

分享

喜欢

收藏

申请转载

...



知乎

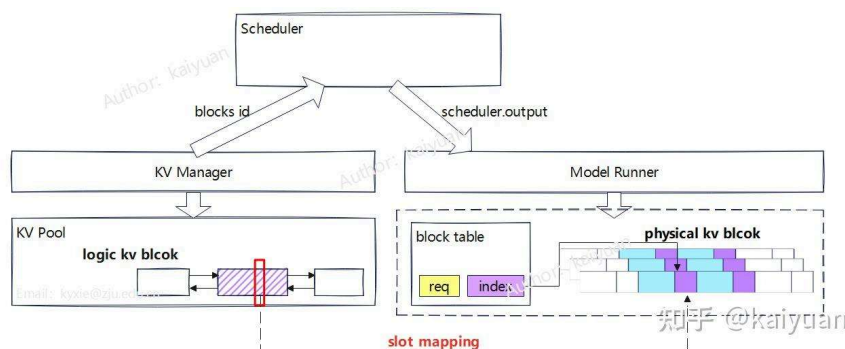
关注 推荐 热榜 专栏 圈子^{New} 付费咨询 知学堂

直答

- 能够减少物理显存的反复申请/释放操作，提升效率；

目前在V1版本中KV cache管理还融合了前缀树的特点，更好地适配了Prefix cache⁺功能。整体的架构如下图所示，分为逻辑层和物理层。逻辑层由KV Manager管理、物理层由Model Runner处理；Scheduler（调度器）作为信息传递的桥梁，衔接了逻辑层与物理层。cache的管理元素包括：池（pool）表(table)、层(layer)、块(block)和槽(slot)。

- **slot**: 为最小管理单元，每个token占一个slot；
- **block**: 为请求分配的基本单位，一个block包含多个slot；
- **pool**: 为逻辑层block的管理合集，通过链表将block数据组织起来；
- **table**: 管理请求与数据的映射表，一个table可包含多个请求的信息。位于物理层；
- **layer**: 一个整体的tensor，拆分成多个blocks使用。对应attention的一个层，所有请求共用；



模块之间运行的关键步骤：

- Scheduler分配资源给请求，通过KV Manager申请逻辑blocks；
- KV Manager把Pool中空闲的blocks选中后给到对应请求；
- 分配好逻辑blocks后Scheduler构建scheduler.output传递给ModelRunner；
- ModelRunner为每条请求创建block table，并生成slot_mapping；
- 计算时把slot_mapping传入attention，就能够从物理KV blocks上面找到所需的数据。

1.2 cache的重复利用

KV cache manager中有个关键功能：重复利用已计算的KV cache，该功能可以降低重复前缀的计算。使用cache对计算量的影响有多大？从计算量的角度来分析一下这个问题：

在prefill阶段，仅考虑Attention（MLA）模块和FFN（MoE）模块的flops计算，通过增加已计算的前缀缓存（prefix cache）长度获得算量的变化。相关公式如下：

回答 62 文章 85 关注者 8,957

已关注

发私信

大家都在搜

换一换

妈祖换人 602 万

热

飞驰人生3 519 万

热

谷爱凌U型场地决赛推迟 477 万

镖人：风起大漠 456 万

热

惊蛰无声 446 万

热

《镖人》票房逆跌 435 万

热

西北华北多地强沙尘暴 367 万

新

贝加尔湖汽车落水7名中国... 298 万

特朗普称对全球加征10%... 273 万

湖北文旅厅认定那艺娜为... 259 万

千问

阿里云百炼 Coding Plan又上新

支持Qwen3.5、kimi2.5、GLM4.7、OpenClaw等6款主流模型

首单特惠

7.9元起

续费套餐限时

5折起

立即体验



知乎

关注

推荐

热榜

专栏

圈子^{New}

付费咨询

知学堂

直答

a1.1> Q的下采样 + Q的上采样

```

q_down_proj = 2 * bs * seq_len * h_dim * q_lora_rank
q_up_proj = 2 * bs * seq_len * q_lora_rank * heads * (qk_head_dim + qk_rope_head_dim)
q_linear = q_down_proj + q_up_proj

```

a1.2> KV的下采样 + 上采样

```

kv_down_proj = 2 * bs * seq_len * h_dim * (kv_lora_rank + qk_rope_head_dim)
kv_up_proj = 2 * bs * heads * (seq_len + cache_len) * kv_lora_rank * (qk_head_dim + v_head_dim)
kv_linear = kv_down_proj + kv_up_proj

```

a1.3> attention计算 + out运算

```

kv_scores = 2 * bs * heads * seq_len * (seq_len + cache_len) * (qk_head_dim + qk_rope_head_dim)
qkv = 2 * bs * heads * seq_len * (seq_len + cache_len) * v_head_dim
out = 2 * bs * seq_len * n_heads * v_head_dim * h_dim
attention = kv_scores + qkv

```

其中seq_len是输入prompt长度, cache_len是prefix cache的存储的长度。

b1> 线性层的flops:

```

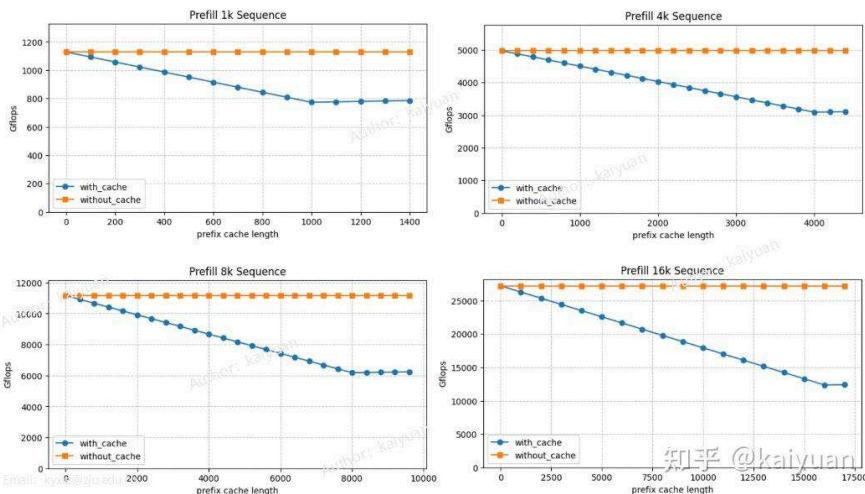
moe_layer_flops = linear_layer_flops * (n_shared_experts + n_activated_experts) + route_flops
其中:
linear_layer_flops = 2 * 3 * bs * seq_len * h_dim * moe_inter_dim
route_flops = 2 * bs * seq_len * h_dim * n_routed_experts

```

知乎 @kaiyuan

选择总长度不同的序列1k、4k、8k和16k分别计算得到如下曲线。通过对比可以知道:

- KV cache匹配命中率越高计算量越小。
- 输入总序列越长, KV cache对计算的影响越大。



计算代码位置: github.com/CalvinXKY/In...

2 逻辑层

2.1 blocks的管理逻辑

KV cache的存储是以block为基本单位组织的。据需求设定block size, 表示block里面可存储的tokens的数量, 用slot来表示token在block中位置。所有的block都在pool里面。

Block的定义: KVCacheBlock, 它是一个双向链表。

```

# 代码位置: vLLM/vLLM/v1/core/kv_cache_utils.py
# KVCacheBlock 定义:

```

```

class KVCacheBlock:

```

```

    """KV-cache block metadata."""

```

```

    # Block ID, ranging from 0 to ...

```

```

    block_id: int

```

4/14



b> 内存释放: request使用完后, 将block送回free队列里面, 如果是cached block, 引用计数-1, 最后的这个block 最先放入队列中, 它可能未被cache;

c> 淘汰策略: 根据LRU策略淘汰队首的block cache, 并从cached blocks字典中去除记录。

基本操作在近期KVCacheManager迭代中有变化, 用以适配新特性比如PD connector、KV cache group等。具体细节参看代码: `vllm/vllm/v1/core/kv_cache_manager.py`

KV cache管理中比较重要的功能:

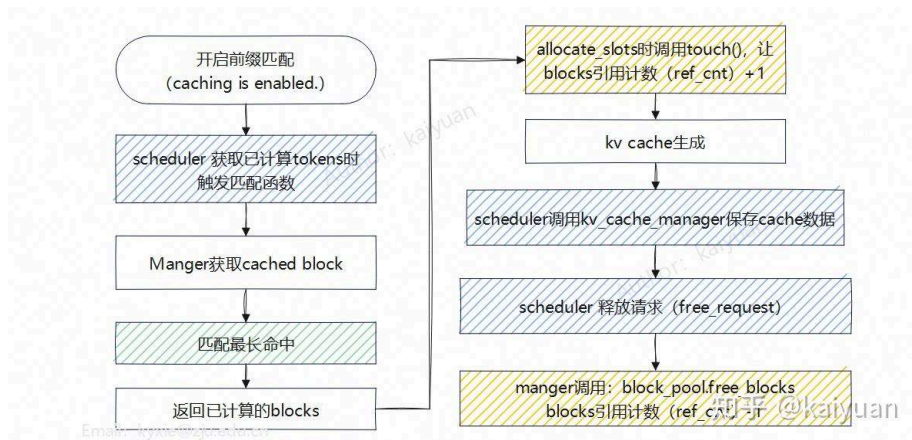
a. Connector: 主要解决跨实例/节点之间传输问题, 建议参看之前的总结,

- 基本原理: [vLLM PD分离KV cache传递机制详解与演进分析](#)
- 相关实践: [0.5x提升:PD分离KV cache传输的实践经验](#)

b. Prefix cache: 前缀匹配, 在prefill阶段, 重复利用已计算的KV cache, 即把保存的KV cache数据(未被淘汰)提供给新请求使用。在创建blocks前, 检查是否有已计算的blocks, 若有则复用这些已计算的blocks。

prefix cache实现由scheduler和kv manager共同参与完成。参数“enable_caching”控制功能的开启与否, 默认开启prefix cache。

主要执行动作: scheduler计算tokens时会触发匹配函数, 然后交由kv manger进行匹配计算并返回已匹配的blocks; 在为tokens申请新slots时, 对block的使用情况进行标记。



相关步骤代码:

```

# scheduler 中调用 kv manager
# vllm/vllm/v1/core/sched/scheduler.py
# 触发匹配计算

# Get already-cached tokens.
if request.num_computed_tokens == 0:
    # Get locally-cached tokens.
    new_computed_blocks, num_new_local_computed_tokens = \
        self.kv_cache_manager.get_computed_blocks(
            request)

# vllm/vllm/v1/core/kv_cache_manager.py
# 进行命中率计算:

max_cache_hit_length = request.num_tokens - 1
computed_blocks, num_new_computed_tokens = (
    self.coordinator.find
  
```



```

new_blocks = self.kv_cache_manager.allocate_slots(
    request,
    num_new_tokens + num_external_computed_tokens,
    num_new_local_computed_tokens,
    new_computed_blocks,
    num_lookahead_tokens=effective_lookahead_tokens,
    delay_cache_blocks=load_kv_async,
    num_encoder_tokens=num_encoder_tokens,
)

```

vllm/vllm/v1/core/kv_cache_manager.py

manger 标记匹配的 blocks:

```

# Touch the computed blocks to make sure they won't be evicted.
if self.enable_caching:
    self.block_pool.touch(new_computed_block_list)

```

vllm/vllm/v1/core/block_pool.py

pool 里面管理的 block 的引用计数 (ref_cnt) +1:

```

def touch(self, blocks: tuple[list[KVCacheBlock], ...]) -> None:
    for blocks_per_group in blocks:
        for block in blocks_per_group:
            # ref_cnt=0 means this block is in the free list (i.e. eviction
            # candidate), so remove it.
            if block.ref_cnt == 0 and not block.is_null:
                self.free_block_queue.remove(block)
            block.ref_cnt += 1

```

vllm/vllm/v1/core/single_type_kv_cache_manager.py

pool 释放请求对应的 blocks:

```

def free(self, request_id: str) -> None:
    """
    Free the blocks for the request.

    Args:
        request_id: The request ID.
    """
    # Default to [] in case a request is freed (aborted) before alloc.
    req_blocks = self.req_to_blocks.pop(request_id, [])

    # Free blocks in reverse order so that the tail blocks are
    # freed first.
    # 此处有个按照逆序来淘汰策略, 但该方式目前看非最佳 (可留言讨论@kaiyuan)
    ordered_blocks = reversed(req_blocks)

    self.block_pool.free_blocks(ordered_blocks)
    self.num_cached_block.pop(request_id, None)

```

vllm/vllm/v1/core/block_pool.py

释放时, 引用计数-1:

```

def free_blocks(self, ordered_blocks: Iterable[KVCacheBlock]) -> None:
    # Materialize the iterable to allow multiple passes.
    blocks_list = list(ordered_blocks)
    for block in blocks_list:
        block.ref_cnt -= 1
    self.free_block_queue.append_n([
        block for block in b:

```

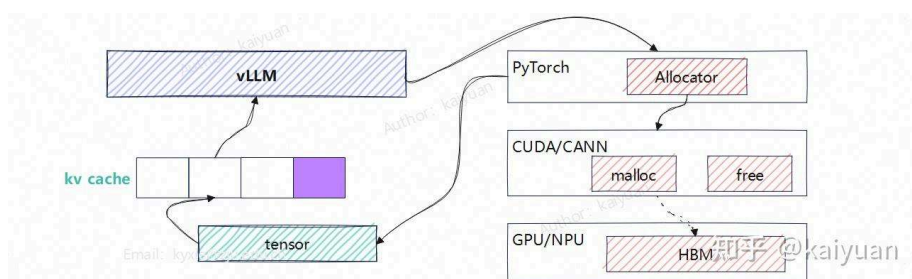


prefix cache数据淘汰的执行逻辑可参看(第2节):

3 应用层 (物理层)

应用层承载实际的KV cache数据，在vLLM框架中也可其视为“物理层”。物理层中单个layer的KV cache，是一个由torch创建的连续tensor。Attention运算时使用该tensor的部分数据。

torch为作为应用层的基础，其有一套独立的显存管理逻辑，该逻辑依赖底层硬件API库。作为框架开发者/使用者，我们不需要太多关注底层逻辑，聚焦到torch之上的内容即可。



显存使用的层级关系

3.1 kv cache的创建

kv cache的创建主要是两个步骤：

- 步骤1，**可用空间计算**。在初始化阶段，确认kv cache可用空间。
- 步骤2，**层的创建**。根据模型特点为每一层分配一个固定的空间。

可用空间计算

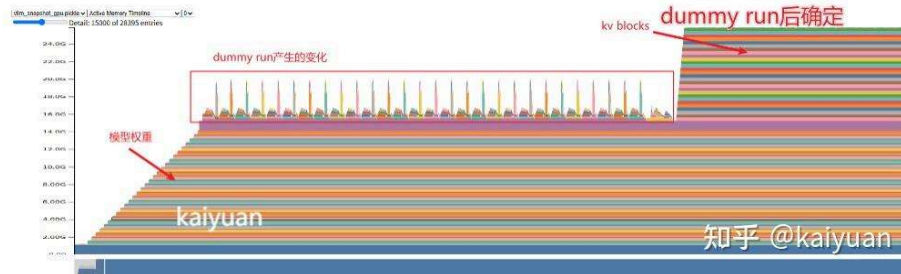
kv cache的可用空间是在预热(dummy run)阶段确定的，在《vLLM整体显存管理》中对这部分内容有介绍。目前可用的空间 (available_gpu_memory) 计算，有两种方式：

```
# vllm/vllm/v1/engine/core.py
# _initialize_kv_caches函数:
if os.environ.get("VLLM_ELASTIC_EP_SCALE_UP_LAUNCH") == "1":
    dp_group = getattr(self, "dp_group", None)
    assert dp_group is not None
    self.available_gpu_memory_for_kv_cache = \
        ParallelConfig.sync_kv_cache_memory_size(dp_group, -1)
    available_gpu_memory = [
        self.available_gpu_memory_for_kv_cache
    ] * len(kv_cache_specs)
else:
    # Profiles the peak memory usage of the model to determine how
    # much memory can be allocated for kv cache.
    available_gpu_memory = (
        self.model_executor.determine_available_memory())
    self.available_gpu_memory_for_kv_cache = \
        available_gpu_memory[0]
```



```
# 代码位置: vllm/vllm/v1/worker/gpu_worker.py determine_available_memory
# 主要函数: determine_available_memory
```

计算满足: $KV\ blocks = 总显存 \times gpu_memory_utilization - (模型显存 + 激活峰值 + 非torch显存)$



blocks的大小计算如下, 其中page size 跟模型结构相关, 不同层的计算可以不一样。

```
# 代码位置: vllm/vllm/v1/core/kv_cache_utils.py
def get_num_blocks(vllm_config: VllmConfig, num_layers: int,
                   available_memory: int, page_size: int) -> int:
    """
    Get the number of kv cache blocks.

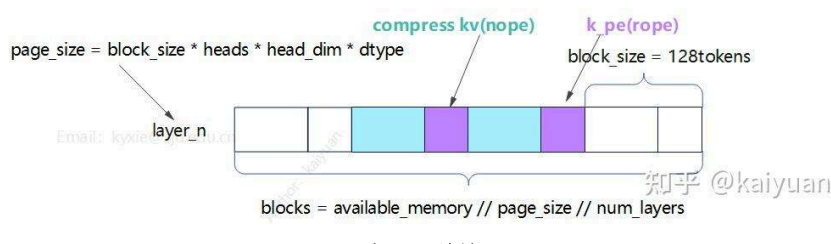
    Args:
        vllm_config: The global VllmConfig
        num_layers: The number of layers
        available_memory: Memory available for KV cache in bytes.
        page_size: The page size of the KV cache.

    """
    num_blocks = int(available_memory // page_size // num_layers)
    num_blocks = max(num_blocks, 0)
    num_blocks = may_override_num_blocks(vllm_config, num_blocks)
    return num_blocks

# 其中同构的attention的page_size计算:
# 代码位置: vllm/vllm/v1/kv_cache_interface.py
def page_size_bytes(self) -> int:
    # For MLA we only store a single latent vector
    coef = 1 if self.use_mla else 2
    return coef * self.block_size * self.num_kv_heads * self.head_size \
        * get_dtype_size(self.dtype)
```

层的创建

物理显存分配是由pytorch完成, 创建的连续tensor视为KV cache所能使用的物理显存。在GPU runner中有一个`_allocate_kv_cache_tensors`函数, 通过`torch.tensor`逐层创建`kv_cache_tensor`。模型每层共用tensor数据, 比如MLA模块+会创建nope和rope模块, nope和rope可以创建在一起, 也可以由两个tensor承载。





nope与rope分开

```
# 代码位置: vLLM/v1/worker/gpu_model_runner.py
kv_cache_raw_tensors: dict[str, torch.Tensor] = {}
kv_cache_raw_tensors: dict[str, torch.Tensor] = {}
for kv_cache_tensor in kv_cache_config.kv_cache_tensors:
    tensor = torch.zeros(kv_cache_tensor.size,
                        dtype=torch.int8,
                        device=self.device)
    for layer_name in kv_cache_tensor.shared_by:
        kv_cache_raw_tensors[layer_name] = tensor
```

3.2 层数据分配

在KV manager中, 为请求分配好逻辑blocks后, 通过scheduler.out传递给model runner, 进而构造request与物理blocks的映射关系; scheduler.out里面还标记了每个请求中已计算完成的tokens。

在model runner中需要完成: scheduler.out信息要转化为PagedAttention所需要的KV数据。

先看一下PagedAttention运算时的数据形态, 以FA为例, 如下所示。根据Attention的一般计算可知, 其输入是: QKV数据、历史的KV数据。

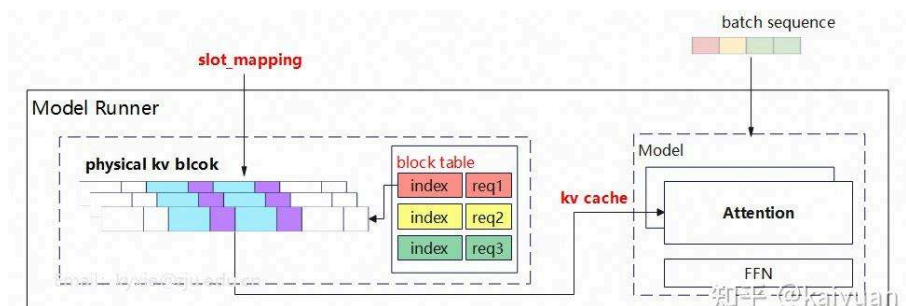
```
# Attention 的输入
Args:
    query: shape = [num_tokens, num_heads, head_size]
    key: shape = [num_tokens, num_kv_heads, head_size]
    value: shape = [num_tokens, num_kv_heads, head_size]
    kv_cache: shape =
        [2, num_blocks, block_size, num_kv_heads, head_size]
    attn_metadata: Metadata for attention.

# 输出:
Returns:
    shape = [num_tokens, num_heads * head_size]
```

其中, query/key/value是当前forward产生的数据, kv_cache参数是整层的数据, 该数据所有请求共用。

接着要明确问题是: Attention拿kv_cache的哪些数据?

在数据读取时仅有block的索引信息是不够的, 还需要知道token在block中的具体位置, 而这个信息存在attn_metadata的slot_mapping中(数据类型为torch.Tensor)。



slot_mapping 如何运作?



先看一个简单示例，从输入到slot_mapping的计算。

Tokenization:

Prompts = ["Hi, I'm kaiyuan", " Do you subscribe InfraTech?"]

Tokens

<s>	Hi	,	I'm	kai	yuan
-----	----	---	-----	-----	------

<s>	Do	you	sub	scribe	Infra	Tech	?
-----	----	-----	-----	--------	-------	------	---

Token IDs

1	3	6	7	9	77
---	---	---	---	---	----

1	25	32	11	15	87	66	10
---	----	----	----	----	----	----	----

数据输出(cat&flatten)

ids = [1, 3, 6, 7, 9, 77, 1, 25, 32, 11, 15, 87, 66, 10]

positions = [0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 6, 7]

KV manager

ids = [1, 3, 6, 7, 9, 77, 1, 25, 32, 11, 15, 87, 66, 10]

positions = [0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5, 6, 7]

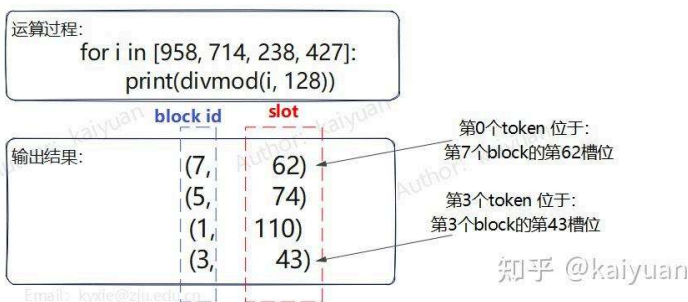
logic kv blkok

block_0	block_1	block_2	block_3	block_4	block_5	block_6
---------	---------	---------	---------	---------	---------	---------

slot_mapping = [11, 12, 13, 14, 15, 16, 91, 92, 93, 94, 95, 96, 97, 98]

前置步骤

具体计算方式：数据进行整除+取余运算，整数表示所在 block id，余数表示在对应的 block id 里面的 slot（槽位）。举个例子，假设 block size=128，slot_mapping=[958, 714, 238, 427]，计算如下：



scheduler是多个请求同时下发，映射关系有多组，slot_mapping如何构造？

vLLM中通过映射表（**block table**）来记录每个请求信息，并处理多请求问题。用BlockTable类实现请求的增、删操作，代码如下（节选）：

```
# 代码位置: vLLM/vLLM/v1/worker/block_table.py
class BlockTable:
    def __init__(
        self,
        block_size: int,
        max_num_reqs: int,
```



```

        device: torch.device,
    ):
# ....
        self.block_table = self._make_buffer(max_num_reqs,
                                              max_num_blocks_per_req,
                                              dtype=torch.int32)
        self.num_blocks_per_row = np.zeros(max_num_reqs, dtype=np.int32)

        self.slot_mapping = self._make_buffer(self.max_num_batched_tokens,
                                              dtype=torch.int64)

    def append_row(
        self,
        block_ids: list[int],
        row_idx: int,
    ) -> None:
        if not block_ids:
            return
        num_blocks = len(block_ids)
        start = self.num_blocks_per_row[row_idx]
        self.num_blocks_per_row[row_idx] += num_blocks
        self.block_table.numpy[row_idx, start:start + num_blocks] = block_ids

# 添加请求:
    def add_row(self, block_ids: list[int], row_idx: int) -> None:
        self.num_blocks_per_row[row_idx] = 0
        self.append_row(block_ids, row_idx)

```

关键参数: **self.block_table**, 其大小等于最大请求数乘以单个请求 blocks 数量最大值。它存储了所有需要运行请求相关的 blocks 信息。

block table 通过 **add_row** 添加请求, 其中 `row_idx` 参数是指传入的请求索引 (`req_index`)。slot_mapping 的生成计算如下 (节选), 由 `compute_slot_mapping` 函数完成。

```

# 代码位置: vLLM/vLLM/v1/worker/block_table.py
    def compute_slot_mapping(self, req_indices: np.ndarray,
                            positions: np.ndarray) -> None:
# .....
        block_table_indices = (req_indices * self.max_num_blocks_per_req +
                               positions // self.block_size) # 找到请求索引的
        block_numbers = self.block_table.numpy.ravel()[block_table_indices] # 展
        block_offsets = positions % self.block_size # 转化为在每个 block 中的位置
        np.add(block_numbers * self.block_size,
               block_offsets,
               out=self.slot_mapping.numpy[:req_indices.shape[0]]) # 构造 slot_ma

```

这个计算实际上就是一个二维转一维的运算。举个例子帮助理解, 假设 `block_size` 大小为 5, `max_num_blocks_per_req` 大小为 4, 索引某个 token 的计算如下:



知乎

关注

推荐

热榜

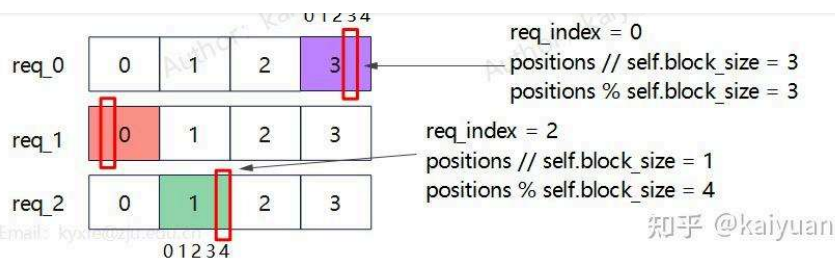
专栏

圈子^{New}

付费咨询

知学堂

直答



接下来, 需要了解req_indices, positions是如何计算出来的?

代码在ModelRunner的_prepare_inputs函数里面, 根据scheduler_output.num_scheduled_tokens的数据来构造req_indices与positions。

注释里面给了一个例子, [2, 5, 3] 表示的是有三个请求, 每个元素表示请求需要的tokens数。其中num_computed_tokens_cpu参数是指已计算过的tokens数量。

```
# 代码位置: vLLM/vLLM/v1/worker/gpu_model_runner.py
# def _prepare_inputs
# ...

req_ids = self.input_batch.req_ids
tokens = [scheduler_output.num_scheduled_tokens[i] for i in req_ids]
num_scheduled_tokens = np.array(tokens, dtype=np.int32) # 调度到每个请求上的
max_num_scheduled_tokens = max(tokens)

# Get request indices.
# E.g., [2, 5, 3] -> [0, 0, 1, 1, 1, 1, 1, 2, 2, 2]
req_indices = np.repeat(self.arange_np[:num_reqs],
                        num_scheduled_tokens)

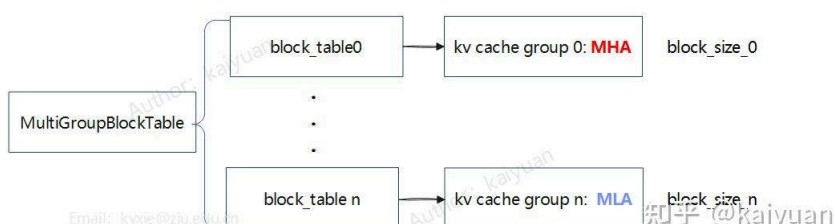
# cu_num_tokens: [2, 5, 3] -> [2, 7, 10]
# arange: [0, 1, 0, 1, 2, 3, 4, 0, 1, 2]
cu_num_tokens, arange = self._get_cumsum_and_arange(
    num_scheduled_tokens)

# Get positions.
positions_np = self.positions_np[:total_num_scheduled_tokens]
np.add(self.input_batch.num_computed_tokens_cpu[req_indices],
       arange,
       out=positions_np)
```

至此分析完了scheduler_output与slot_mapping的计算逻辑, 还有两个细节点:

i> KV cache group

为了适配异构形态、或者不同Attention模块 (MHA/MLA/GQA/slide等) 的kv cache混合使用设计了kv cache group概念, 对应的block size可以设置不同, 用不同的block table来管理每个KV cache group的映射关系。不同KV cache group之间协同工作由KVCacheCoordinator管理。



- github.com/vllm-project...
- github.com/vllm-project...

ii> DCP (Decode Context Parallel)

为了减少KV cache的冗余存储，开启Attention序列并行时，可根据CP的数量让不同设备存储KV cache的部分数据。

当前计算逻辑单个token的存储位置满足： $\text{rank_id} = \text{token_idx} \% \text{cp_world_size}$ 。

关键细节介绍到这里，其它内容参考源码：

新模型在不断推出，KV cache管理也在持续迭代，下一步的方向：

- **模型的适配**: 当模型中Attention结构出现改进时，目前代码修改涉及逻辑层和应用层，这种方式很显然是高成本的，所以需要一种更加解耦降低修改量。
- **池化应用**: 本地池化和跨节点的池化功能应该成为一种标配。
- **传输优化**: 全掩盖的传输方式、不影响指标(TTFT/ITL) 的传输方式的探索。

更多LLM推理内容：

欢迎评论、点赞、关注 @kaiyuan

所属专栏 · 昨天 11:53 更新



LLM 推理基础与框架

 **kaiyuan**  大模型话题下的优秀答主

43 篇内容 · 7172 赞同

订阅

最热内容 · vLLM PD分离方案浅析

编辑于 2026-01-27 16:00 · 中国香港

LLM推理 大模型 AI大模型



阿里云百炼首单低至 7.9 元/月，一键开启 AI 编程新体…
Qwen 3.5 发布 coding plan 2，支持 OpenClaw、kimi2.5、GLM4.7 等 6 款主流模型、首单 7.9 元/月、续费特惠享 5 折； [查看详情](#)

阿里云 的广告



理性发言，友善互动

请问博主这些示意图都是用什么软件画的呢🤔

02-03 · 加拿大

回复 喜欢



kaiyuan 作者

[draw.io](#)和亿图，都用

02-08 · 中国香港

回复 喜欢



KINGSH

GEMM对两个矩阵进行操作，那有一个疑问：vllm使用paged attention导致同一个request的kv cache可能在物理内存中是离散存储的，那后面GEMM运算时，需要把kv cache取出，拼到连续的物理内存空间再输入给GEMM吗

2025-11-24 · 北京

回复 喜欢



kaiyuan 作者

如果你说的拼接是采用会产生HBM临时变量的操作的话，答案是：否。kv cache是块，并非完全的单个数据离散。目前flash的方式都是tile操作，恰好读的就是块数据。

2025-11-25 · 中国香港

回复 喜欢



吹不断的苇草

请问一下KV Cache Manager这里是如何判断复用的数据的，比如cached_block_hash_to_block中命中说明可以复用，不命中会evict；但是这里只记录了block的逻辑面信息；数据面真正的数据怎么区分数据会被丢弃还是会被复用。

2025-10-25 · 广东

回复 喜欢



kaiyuan 作者

吹不断的苇草

是metadata带入的信息，从哪个位置开始写数据。slot mapping只是用来找数据的，所有位置都要有。

2025-10-29 · 浙江

回复 喜欢



吹不断的苇草

我理解一下：scheduler知道了num_scheduled_tokens；这些tokens里对应的position会在_prepare_inputs经过转换获取slot mapping，而前面已经缓存的token已经分配好了block，但是不会包含在slot mapping中，这样计算时自动跳过了物理空间的复写？

2025-10-28 · 广东

回复 喜欢

查看全部 6 条回复



星际迷航

用vllm部署了3个模型在6块4090上跑，利用率很高，同时支持100个用户

2025-10-22 · 云南

回复 喜欢

推荐阅读

vLLM 多模态cache缓存机制分析

最近在梳理vllm的多模态相关feature，发现cache的管理机制比较复杂，所以做了分析和记录。概述vLLM实现了一套复杂的多层缓存系统来优化多模态推理性能。这些缓存机制协同工作，在不同的...
黑白

一文读懂 KV Cache：大模型生成速度的秘密武器！（含数学...

如果你最近在研究 ChatGPT、GPT-4、Claude 等大语言模型，或者正在部署推理服务，肯定会听说过「KV Cache」。它究竟是什么？为什么对生成速度有巨大影响？本文将 intuitively 带你理解它的...
Yixin



大模型 KV Cache 节省神器 MLA 学习笔记（包含推理时...

BBuf

发表于 Giant...

大模型推理加速：KV Cache 和 GQA

打个小广告👉，知乎专栏《大模型前沿应用》的内容已经收录在新书《揭秘大模型：从原理到实战》中。感兴趣的朋友可以购买，多谢支持！❤️ KV Cache 是大模型推理性能优化的一个常用技术，...
绝密伏击